Prototype

Matthew Goldenberg, Maria Wood

2026-11-02

Initial Generation of the Virginia Dataset.

Install Dependencies

This code requires the following packages to run successfully: tidyverse, data.table, sf, tigris. If this is your first time using R and this code, switch eval: false to be eval: true and run the code to install these packages. Then switch this back to eval: false.

```
# Install any missing packages once if needed:
install.packages(c("tidyverse", "data.table", "sf", "tigris"))
```

Loads the needed R Libraries

```
library(tidyverse)
library(data.table)
library(sf)
library(tigris)
options(tigris_use_cache = TRUE)
```

How to correctly download the initial data.

Data Costs: Download Size: ~653 MB (ZIP file) **Extracted Size:** ~2.84 GB (CSV file) **Time Cost (extraction and a full render/run):** ~2–4 minutes (depending on internet speed and extraction time). **Location URL:** https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents?resource=download **Location to Save:** create a folder called Data and place this document there. **Todo before Run:** In the first code chunk make eval: false into eval: true to download the full initial file. Then turn back to false.

To correctly generate the initial data set to use in the follow up charts and graphs follow these directions.

1. Create a folder called Data and a Folder Called Scripts.
2. Capitalization Matters, both folders should start with a capital letter.
3. These two folders should be subfolders in the same parent folder and the parent folder can be called whatever you want the parent folder to be called.
4. Save this QMD file in the folder called Scripts.
5. Navigate to this website: https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents?resource=download
6. Click download in the upper right hand corner of the website.
7. Click download dataset as zip.
8. Click the file that opens.
9. Click extract all.
10. Extract it to the folder you created labeled Data so that the code can work.

Extraction of Virginia from the full dataset.

The below code extracts from the full data set file those rows that have the value under "State" = VA. This is in an effort to try to get a Virginia data set only to draw graphs and run code on.

```
# Stop with a clear message if the raw file is absent
if (!file.exists("../Data/US_Accidents_March23.csv")) {
  stop("Source file missing. Please download US_Accidents_March23.csv (~2.84GB) from the Kagg
}

# Bring Full table in
dffull <- fread("../Data/US_Accidents_March23.csv") %>%
  as_tibble()

# Filter full table as just state
dfva <- dffull %>%
  filter(State == "VA")
# Write VA Only Dataset------------------------------
write_csv(dfva, "../Data/Virginia_Accidents.csv")
```

Validation Code of Virginia Data Set Against Full Data Set

What your verification established about that first output. A defect you found and corrected, or a check it survived — both are complete answers. Name the check, and name what it could not settle.

The code below shows that the Virginia data survives two checks.

1. Does the Virginia only data set have the same count of columns as the data set in Kaggle?

a. This protects against the authors changing the data set on Kaggle by adding columns that aren't represented in the data set that the code is run. The idea is that this prompts a re-download of the original data set.

2. Does the Virginia only data set have the types of VA as the full data set?

a. This protects against the authors using a different code for VA in the update of the full data set. If the authors use VA, Virginia, or any blanks this could impact our future graphs.

```
# Verify that the data brought in has the same dimensions as the full dataset
check_col <- function(x, y){
  stopifnot(ncol(x) == ncol(y))
  invisible(TRUE)
}

# create corruption
set.seed(123)
dfvacor <- dfva %>%
  mutate(corcol = rnorm(nrow(dfva), mean = 50, sd = 10))

# Verify works on original and fails on corrupted.
c(passes_originalnew = !inherits(try(check_col(dffull, dfva),
                                     silent = TRUE), "try-error"),
  rejects_corruptnew =  inherits(try(check_col(dffull, dfvacor),
                                     silent = TRUE), "try-error"))
```

```
passes_originalnew rejects_corruptnew
              TRUE                TRUE
```

```
#---------------------
### --- Verify ALL Virginia observations were pulled in
#(Completeness & Alias Check)
check_va_complete <- function(full_states, y) {
  # 1. Count exact "VA" matches
  expected_n <- sum(full_states == "VA", na.rm = TRUE)

  # 2. Extract unique states first
  unique_states <- unique(full_states)
```

```
  clean_unique <- gsub("\\.", "", trimws(toupper(unique_states)))

  # 3. Find which unique state values match "VA" or "VIRGINIA" or other versions
  # and count their total occurrences
  va_variants <- unique_states[clean_unique %in% c("VA", "VIRGINIA")]
  clean_n <- sum(full_states %in% va_variants, na.rm = TRUE)

  stopifnot(
    !anyNA(y$State),
    all(y$State == "VA"),
    nrow(y) == expected_n,
    expected_n == clean_n
  )
  invisible(TRUE)
}

# Create corruption 1: Simulate dropping 10 rows from dfva
dfva_incomplete <- dfva %>%
  slice(-c(1:10))

# Create corruption 2: Use a TINY mock dataset instead of
#duplicating dffull to save RAM
mock_full_states <- c("MD", "NY", "VA", "VA", "Virginia", "va")
mock_dfva_bad <- data.frame(State = c("VA", "VA"))
# Misses the "Virginia" and "va" variants

# Verify works on original and fails on both corruptions
c(passes_original        = !inherits(try(check_va_complete(dffull$State, dfva),
                                         silent = TRUE), "try-error"),
  rejects_dropped_rows   =  inherits(try(check_va_complete(dffull$State, dfva_incomplete),
                                         silent = TRUE), "try-error"),
  rejects_virginia_spell =  inherits(try(check_va_complete(mock_full_states, mock_dfva_bad),
                                         silent = TRUE), "try-error"))
```

```
      passes_original   rejects_dropped_rows rejects_virginia_spell
                 TRUE                   TRUE                   TRUE
```

```
# Remove Unneeded memory from R Global Environment.----------------------
rm(dfvacor, dffull, dfva_incomplete, mock_full_states, mock_dfva_bad)
```

Attempting Join of the Virginia Data Set with the US Census Bureau data.

This code realizes that the Virginia Data Set by itself will unfairly bias results to the more populous counties as the "most accidents" are likely to occur in the biggest counties by virtue of the fact that there are just simply more people on the road. So we thought it might be helpful to try to link the current data set using a join to a Census Bureau population data set for Virginia counties.We can then weight our count of accidents by the population in each county to show a rate instead of a raw count of accidents per county.

Our data set from kaggle offers two possibilities to join to a population data set. First, our data set has "Counties" as a column variable. It should be possible, but complicated, to do text editing and link this column to the Census data. Second, our data set has geolocation data such as longitude and latitude of the accident. Because analyzing counties was important to us with our question, we choose the first method as it seemed to have a stronger tie to the original county column in the data set and the exact geo-location of the accident may not be a 1-1 tie to the official US Census Bureau county name given in our Kaggle data set.Then we used the geolocation data set as a verification check with the idea that if there is a low disagreement between what the geolocated data of the accident would say regarding the location of the accident, and the county name, then we can trust the population values we are drawing from the Census Bureau. The verification check also made sure using anti_join, that we were not missing any Virginia populations.

Finally, at the end, un-needed data frames were removed from the R image for efficient memory storage.

What your verification established about that first output. A defect you found and corrected, or a check it survived — both are complete answers. Name the check, and name what it could not settle.

The verification check found that the Virginia data set survived a test to make sure there was no un-joined counties from the Census Bureau data set with 0 unmatched census keys. When looking at the comparison between approaches (geolocation codes approach versus performing the join by county names), the mismatch between approaches was small. As a result of this mismatch, this check could not really settle if location would have been a better then longitude and latitude for judging where the accident occurred and what population value to assign to each accident.

```
# ============================================================================
# FETCH & CLEAN OFFICIAL US CENSUS POPULATION DATA FOR VIRGINIA
# ============================================================================

dfva<-read.csv("../Data/Virginia_Accidents.csv")

census_url <- "https://www2.census.gov/programs-surveys/popest/datasets/2020-2023/counties/to
```

```r
va_pop <- read_csv(
  census_url,
  locale = locale(encoding = "latin1"),
  show_col_types = FALSE
) %>%
  filter(STNAME == "Virginia", SUMLEV == "050") %>%
  # 050 = All 133 VA Counties & Independent Cities
  transmute(
    GEOID       = paste0(STATE, COUNTY),
    census_name = str_squish(CTYNAME),
    census_key  = str_to_lower(census_name),
    population  = POPESTIMATE2023
  )

# ============================================================================
#  HARMONIZE VIRGINIA COUNTY & INDEPENDENT CITY NAMES
# ============================================================================
# The 34 VA independent cities that do NOT share a name with a surrounding county
unambiguous_va_cities <- c(
  "alexandria", "bristol", "buena vista", "charlottesville", "chesapeake",
  "colonial heights", "covington", "danville", "emporia", "falls church",
  "fredericksburg", "galax", "hampton", "harrisonburg", "hopewell",
  "lexington", "lynchburg", "manassas", "manassas park", "martinsville",
  "newport news", "norfolk", "norton", "petersburg", "poquoson",
  "portsmouth", "radford", "salem", "staunton", "suffolk",
  "virginia beach", "waynesboro", "williamsburg", "winchester"
)

dfva_clean <- dfva %>%
  mutate(
    # 1. Standardize encoding,
    #remove parentheses around "(city)" or "(county)", and lowercase
    cnty_raw   = str_to_lower(str_squish(iconv(County, to = "UTF-8", sub = ""))),
    cnty_clean = str_squish(str_replace_all(cnty_raw, "[()]", "")),

    # 2. Extract base name by stripping trailing " county" or " city"
    cnty_base  = str_remove(cnty_clean, "\\s+(county|city)$"),

    # 3. Map every jurisdiction 1-to-1 from the County column to the Census CTYNAME format
    census_key = case_when(
      # Fix the 2 Virginia counties that literally have "City" in their county name
      cnty_base %in% c("james", "james city") ~ "james city county",
```

```
      cnty_base %in% c("charles", "charles city") ~ "charles city county",

      # Bedford city merged into Bedford County in 2013
      cnty_base == "bedford" ~ "bedford county",

      # Resolve the 4 VA County vs. Independent City collisions
        #(Fairfax, Roanoke, Richmond, Franklin)
      # Because raw independent cities are labeled
        #"(city)" or "city", keep them as "city",
        # and map plain "Fairfax" / "Roanoke" / "Richmond" /
        #"Franklin" directly to "county"
      cnty_clean %in% c("fairfax city", "roanoke city",
                        "richmond city", "franklin city") ~ cnty_clean,
      str_detect(cnty_clean, "^city of (fairfax|roanoke|richmond|franklin)$") ~ paste0(str_re
      cnty_base %in% c("fairfax", "roanoke", "richmond", "franklin")
      ~ paste0(cnty_base, " county"),

      # Append " city" to the 34 unambiguous independent cities
      cnty_base %in% unambiguous_va_cities
      ~ paste0(cnty_base, " city"),

      # Append " county" to all remaining Virginia counties
      !is.na(cnty_base) & cnty_base != "" ~ paste0(cnty_base, " county"),
      TRUE ~ NA_character_
    )
  )



#---------------------------------

# Verify 100% match rate against Census
unmatched <- dfva_clean %>%
  anti_join(va_pop, by = "census_key") %>%
  count(County, City, census_key, sort = TRUE)

print(unmatched)
```

```
[1] County     City       census_key n
<0 rows> (or 0-length row.names)
```

```r
# 2. Left join population data onto the accident records
  # and remove helper columns
dfva_joined <- dfva_clean %>%
  left_join(va_pop, by = "census_key") %>%
  select(-cnty_raw, -cnty_clean, -cnty_base)

# ============================================================================
# STEP 5 SPATIAL AUDIT COMPARING COUNTY TEXT VS. GPS COORDINATES
# ============================================================================


# cb = FALSE uses full legal boundaries
#(including bridges over water in Hampton Roads/Arlington)
va_polygons <- counties(state = "VA", cb = FALSE, year = 2022) %>%
  transmute(
    spatial_GEOID = GEOID,
    spatial_name  = NAMELSAD,
    spatial_key   = str_to_lower(str_squish(NAMELSAD))
  )

dfva_audit <- dfva_clean %>%
  filter(!is.na(Start_Lng), !is.na(Start_Lat)) %>%
  st_as_sf(coords = c("Start_Lng", "Start_Lat"),
           crs = 4326, remove = FALSE) %>%
  st_transform(st_crs(va_polygons)) %>%
  st_join(va_polygons, join = st_within) %>%
  st_drop_geometry()

audit_summary <- dfva_audit %>%
  summarise(
    total_rows       = n(),
    exact_matches    = sum(census_key == spatial_key, na.rm = TRUE),
    mismatches       = sum(census_key != spatial_key, na.rm = TRUE),
    outside_va_poly  = sum(is.na(spatial_key)),
    match_percentage = round((exact_matches / total_rows) * 100, 2)
  )


mismatch_breakdown <- dfva_audit %>%
  filter(census_key != spatial_key | is.na(spatial_key)) %>%
  count(County, City, text_assigned = census_key,
        gps_assigned = spatial_key, sort = TRUE)
```

```
## Write the joined dataset
# Save full row-level Virginia dataset with Census population attached
write_csv(dfva_joined, "../Data/Virginia_Accidents_With_Pop.csv")
##Remove Unneeded Memory from R image-----------------------------------
rm(dfva, dfva_clean, dfva_audit, dfva_joined)
```

Validation of Analysis Data Set only

Since accident Severity is a main portion of our analysis, this validation check looks at accident Severity and makes sure that there are no NA's or values off the expected accident Severity scale of 1-4. This tests the data dictionary on Kaggle for this column where it describes Severity as:

Shows the severity of the accident, a number between 1 and 4, where 1 indicates the least impact on traffic (i.e., short delay as a result of the accident) and 4 indicates a significant impact on traffic (i.e., long delay).

What your verification established about that first output. A defect you found and corrected, or a check it survived — both are complete answers. Name the check, and name what it could not settle.

The check on severity survived, all observations in the Virginia Data Set have Severity 1-4 and no NA's. Severity is defined as delay, but delay can happen in an accident for items that do not pertain to the Severity of the accident. For example, a remote location could cause a higher "delay" because its harder to get accident clean up equipment to that location. Thus, this check could not measure whether severity is a good measure of challenge in an accident.

```
dfva_joined <- read.csv("../Data/Virginia_Accidents_With_Pop.csv")

#Check Accident Severity
check_severity <- function(x){
  stopifnot(sum(unique(x$Severity))<=10,
            !any(is.na(x$Severity)))
  invisible(TRUE)
}

#data corruption insertion of severity 5
dfvacorrupt<-data.frame(Severity = 1:5, Random = 1:5)
#data corruption, insertion of NA values.
dfvacorrupt1<-data.frame(Severity = as.numeric(c(1,2,"",4, 4)), Random = 1:5)
```

```
#Checks Severity
c(passes_original        = !inherits(try(check_severity(dfva_joined),
                                         silent = TRUE), "try-error"),
  rejects_corrupt   =  inherits(try(check_severity(dfvacorrupt),
                                         silent = TRUE), "try-error"))
```

```
passes_original rejects_corrupt
           TRUE            TRUE
```

```
#Checks NA
c(passes_original        = !inherits(try(check_severity(dfva_joined),
                                         silent = TRUE), "try-error"),
  rejects_corrupt   =  inherits(try(check_severity(dfvacorrupt1),
                                         silent = TRUE), "try-error"))
```

```
passes_original rejects_corrupt
           TRUE            TRUE
```

What your verification established about that first output. A defect you found and corrected, or a check it survived — both are complete answers. Name the check, and name what it could not settle.

Finally, because we might want to draw a time graph or animation in the dashboard, we decided to validate start and end time of the accident as well to make sure there are no data errors (end time coming before start time) and that every accident has an accident timing.

Our data survived this check, start time in the data set is always before the end time of the data set and that there are no missing values. End time is defined in the data set by:

"Shows end time of the accident in local time zone. End time here refers to when the impact of accident on traffic flow was dismissed."

This data check can not establish whether "when traffic flow was dismissed" is an accurate judgment of end time as "when traffic flow was dismissed is somewhat ambiguous.

```
dfva_joined$Start_Time<- as.POSIXct(
    dfva_joined$Start_Time,
    format = "%Y-%m-%dT%H:%M:%OSZ",
    tz = "UTC"
  )
dfva_joined$End_Time <- as.POSIXct(
    dfva_joined$End_Time,
```

```
    format = "%Y-%m-%dT%H:%M:%OSZ",
    tz = "UTC")

#Check Timing
check_timing <- function(x){
  stopifnot(x$Start_Time <= x$End_Time,
            !any(is.na(x$Start_time)),
            !any(is.na(x$End_time)))
  invisible(TRUE)
}
corrupttiming<- data.frame(Start_Time = as.POSIXct(
    "2016-11-30T16:52:00Z",
    format = "%Y-%m-%dT%H:%M:%OSZ",
    tz = "UTC"
  ), End_Time = as.POSIXct(
    "2016-11-30T12:52:00Z",
    format = "%Y-%m-%dT%H:%M:%OSZ",
    tz = "UTC"
  ), random = 1:5)

#Checks Timing
c(passes_original        = !inherits(try(check_timing(dfva_joined),
                                         silent = TRUE), "try-error"),
  rejects_corrupt   =  inherits(try(check_timing(corrupttiming),
                                         silent = TRUE), "try-error"))
```

```
passes_original rejects_corrupt
           TRUE            TRUE
```

Graphs and Summary Tables

The Question We Are Investigating From M1

Which five Virginia counties experience the highest frequency of severe traffic accidents, and which weather conditions are most strongly related to these severe accidents?

The estimand. What quantity your artifact actually reports, and what it does not identify.

Our artifact reports the top five counties with the most severe accidents, the top five counties with the highest severe accidents to population ratio, and the top ten weather conditions with

the highest severe accidents to total accidents percentage. It does not identify whether the accidents are related to time variables or in/out of state residents. It also does not report the top weather conditions by county for severe accidents.

We plan to use these graphs to create the first page of a dashboard. The purpose of this page is to give the governor an immediate understanding of the worst effected counties. We also plan to have this page have a map of Virginia with the worst effected counties filtered by year. Later pages will include a more in depth look at each individual county. Each in depth look will investigate different conditions related to severe accidents.

```
###---------------------- Graphing
dfva_joined <- read.csv("../Data/Virginia_Accidents_With_Pop.csv")

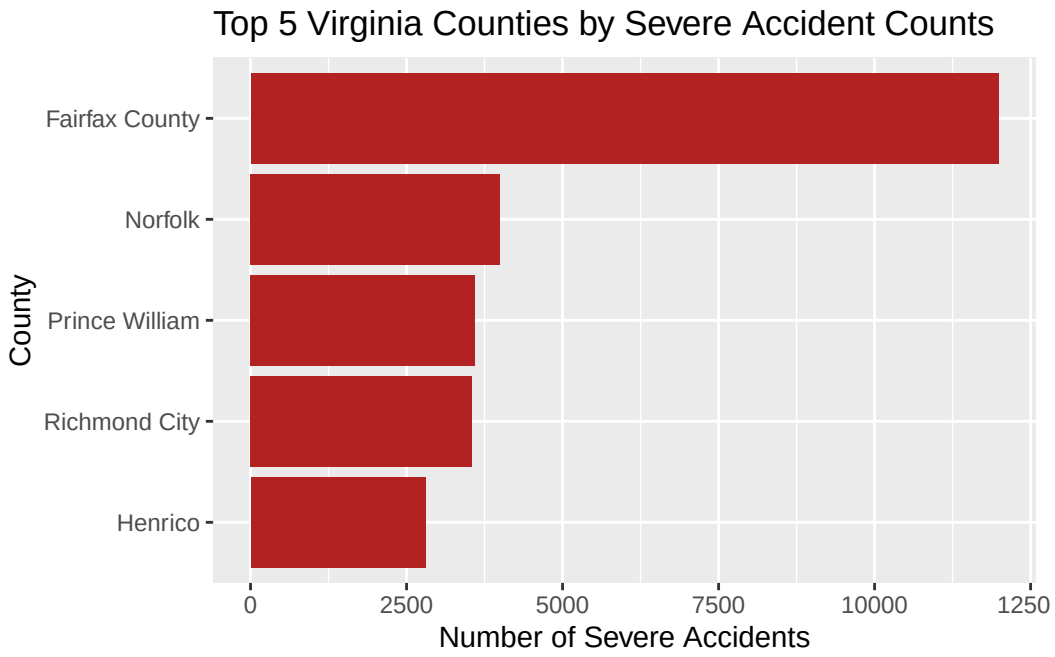
acc<-dfva_joined

acc <- acc %>% mutate(severe = Severity >= 3)

# Severe Accident Counts by County

county_summary <- acc %>%
  filter(!is.na(County)) %>%
  group_by(County) %>%
  summarise(
    severe_accidents = sum(severe == TRUE, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(severe_accidents))

county_summary %>%
  slice_head(n = 5) %>%
  ggplot(
    aes(x = reorder(County, severe_accidents),
        y = severe_accidents)
  ) +
  geom_col(fill = "firebrick") +
  coord_flip() +
  labs(
    title = "Top 5 Virginia Counties by Severe Accident Counts",
    x = "County",
    y = "Number of Severe Accidents"
  )
```



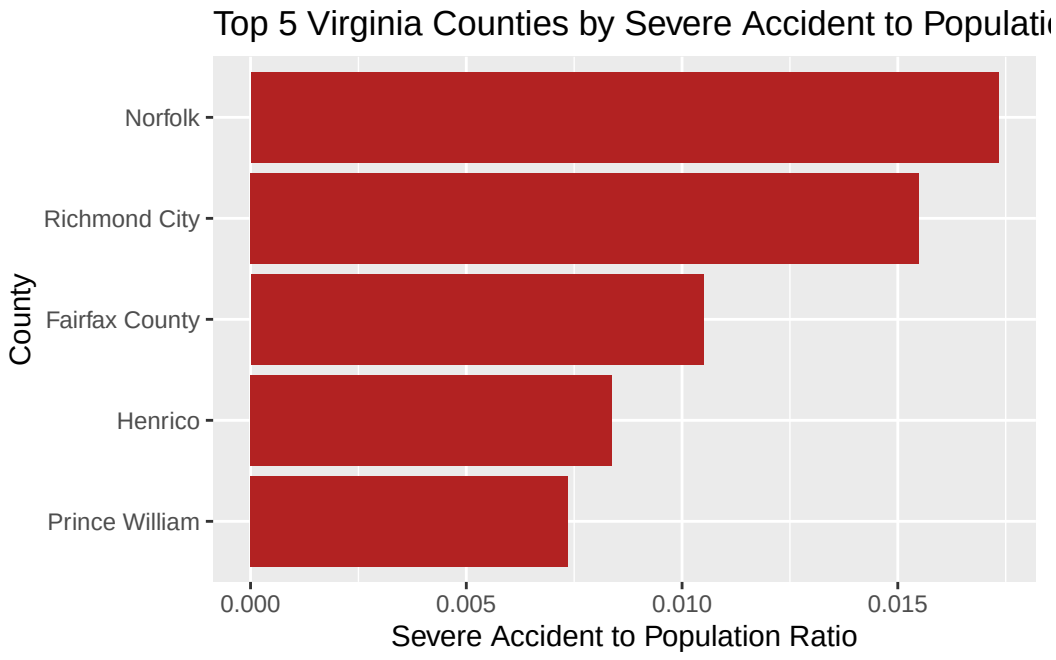

```r
# Severe Accidents to Population Ratio by County

county_summary_p <- acc %>%
  filter(!is.na(County)) %>%
  group_by(County) %>%
  summarise(
    severe_accidents = sum(severe == TRUE, na.rm = TRUE),
    population = first(population),
    severe_ratio = severe_accidents / population,
    .groups = "drop"
  ) %>%
  slice_max(severe_accidents, n=10)%>%
  arrange(desc(severe_accidents))
knitr::kable(county_summary_p, digits = 3)
```

County	severe_accidents	population	severe_ratio
Fairfax County	11986	1141878	0.010
Norfolk	4003	230930	0.017
Prince William	3600	489640	0.007
Richmond City	3552	229247	0.015
Henrico	2804	334760	0.008

County	severe_accidents	population	severe_ratio
Arlington	2120	234162	0.009
Chesapeake	1945	253886	0.008
Virginia Beach	1841	453649	0.004
Augusta	1660	78247	0.021
Stafford	1571	165428	0.009

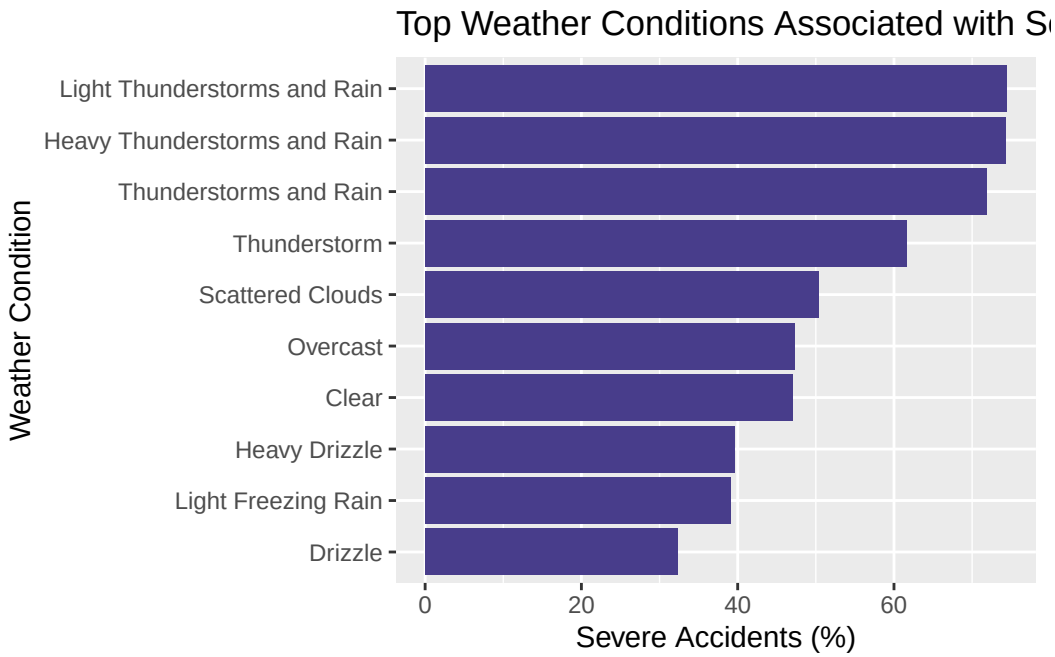
```
county_summary_p %>%
  slice_head(n = 5) %>%
  ggplot(
    aes(x = reorder(County, severe_ratio),
        y = severe_ratio)
  ) +
  geom_col(fill = "firebrick") +
  coord_flip() +
  labs(
    title = "Top 5 Virginia Counties by Severe Accident to Population Ratio",
    x = "County",
    y = "Severe Accident to Population Ratio"
  )
```



```r
# Severe Weather Conditions Percentage

weather_summary <- acc %>%
  filter(!is.na(Weather_Condition)) %>%
  group_by(Weather_Condition) %>%
  summarise(
    total_accidents = n(),
    severe_accidents = sum(severe, na.rm = TRUE),
    severe_percent = severe_accidents / total_accidents * 100,
    .groups = "drop"
  ) %>%
  filter(total_accidents >= 30) %>%
  slice_max(severe_percent, n = 10)

ggplot(
  weather_summary,
  aes(x = reorder(Weather_Condition, severe_percent),
      y = severe_percent)
) +
  geom_col(fill = "darkslateblue") +
  coord_flip() +
  labs(
    title = "Top Weather Conditions Associated with Severe Accidents",
    x = "Weather Condition",
    y = "Severe Accidents (%)"
  )
```



AI Tracker

Task	Tool/model & date	AI contribution	Human revision	Verification performed	Residual concern
AI to Help Set Up GitHub Repo	Gemini Pro. 10/19/2026	Step by Step Directions For How to Set Up the Repo Successfully	NA	NA	NA
Forgot how to add a Column to a DataFrame so I could corrupt it for a check	Gemini Pro, 10/25/2026	Suggested to use dplyr mutate function to add the column	Successfully executed dpylr mutate to create corrupted data check.	NA	Only checks if columns are the same count of columns. Doesn't check if any have changed since the last update.

Task	Tool/model & date	AI contribution	Human revision	Verification performed	Residual concern
Wasn't really sure how to create the check successfully to make sure all VA records from the full dataset were successfully brought over to the VA one. Wanted it to adjust for abbreviations, acronyms etc.So figuring out the string combos, everything to check, etc. got super complicated.	Gemini Pro, 10/25/2026	Created full check.	copied code in	NA	NA
Full check was taking a while to render so wanted to look into whether there was a simpler easier method	Gemini Pro 10/29/26	AI taught me how I could simplify the corrupt data frames and data itself so the validations run on a smaller subset	Used the AI provided Code	Ran Successfully	None

Task	Tool/model & date	AI contribution	Human revision	Verification performed	Residual concern
Join and normalize Virginia traffic accident data with U.S. Census population totals in R	Gemini Pro, October 29, 2026	Developed tidyverse and Base R scripts to download Census data, harmonize Virginia's independent city vs. county naming collisions, and perform a spatial (sf/tigris) vs. text geocoding audit.	User provided dataset structure context, screenshot evidence of (city) formatting and James City/Charles City errors, and a spatial mismatch audit table, driving iterative regex and rule refinement.	AI utilized the ds_python_interpreter to mathematically verify the row sums (1,737 rows) and bidirectional highway border splits from the user's uploaded mismatch screenshot.	Inherent dataset geocoding biases remain: mailing address bleed (e.g., Fairfax accidents labeled Alexandria), enclave city misattribution (Salem vs. Roanoke County), and lack of adjustment for commuter/through-traffic volume.
Forgot how to coerce time variables to non-character in R	Gemini Pro 11/1/2026	Suggested the code in my timing check	Used as stated in the AI	Verification in part conducted with generating corrupt data to make sure the check worked.	NA

|Used AI to include population in graphs|ChatGPT November 2026|AI included `population = first(population)`|NA|Copied the code and it worked. |NA|

Individual Tracker

Date	Who Did It	What Was Produced	How Was It Verified
10/29/2026	Matthew	Created QMD	We reviewed it together
10/28/2026	Matthew	Created Validation Checks	We reviewed it together
10/27/2026	Matthew	Joined Dataset to US Census Bureau	We reviewed it together
10/30/2026	Maria	Created Graphs	We reviewed it together
11/2/2026	Maria	Included Graphs on QMD	We reviewed it together